Understanding Timing Analysis of D Flip-Flop using OpenSTA

INTRODUCTION:

In this lab, students will learn how to perform static timing analysis on a synthesized D flip flop using OpenSTA. They will practice importing the gate-level netlist from Yosys, loading timing constraints, linking standard cell libraries, and generating detailed timing reports. This exercise demonstrates how timing is verified for hardware-efficient logic at the signoff level.

APPLICATION:

A D flip-flop is widely used in digital electronics as a fundamental building block for data storage and synchronization. It stores a single bit of data, capturing the input value on the clock's active edge and holding it until the next clock event. Common applications include creating registers, memory units, and counters. They are also essential in designing state machines and sequential circuits, providing stable timing control and eliminating unpredictable states present in simpler flip-flop types.

OBJECTIVE:

After completing this lab, you will be able to:

- Import a gate-level netlist and standard cell library into OpenSTA (OSU018 lib).
- Load and apply timing constraints (e.g., clock, input/output delays) using SDC format.
- Perform static timing analysis to check setup and hold margins across the design.
- Understand timing reports, including slack values, critical paths, and violation detection.
- Interpret timing analysis results to guide optimization of digital designs for timing closure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

1.1: Create a directory for your lab: `mkdir <project_name>`

1.2: Open lab directory: `cd <project_name>`

1.3: Create directory for project: `mkdir d_ff`

1.4: Open project directory: `cd d_ff`

Step 2: Create Design file

2.1: Open gvim editor for design with command: `gvim d_ff.v`

RTL Design Code:

```
module d_ff ( input clk,
                  input rst, input d,
                  output reg q );
always @ (posedge clk) begin
if (rst)
q <= 0;
else
q <= q;
end
endmodule
```

2.2: save the file using command- `:wq`

Step 3: Create .sdc file

3.1: Open gvim editor for the SDC with command: `gvim d_ff.sdc`

For 10ns:

```
create_clock -period 10 -name clk [get_ports clk]
set_input_delay 5 -min -rise [get_ports clk] -clock clk
set_input_delay 5 -max -fall [get_ports rst] -clock clk
set_input_transition 5 -min -rise [get_ports rst] -clock clk
set_input_transition 5 -max -fall [get_ports clk] -clock clk
set_load -pin_load 4 [get_ports out]
set_output_delay 2 -min -rise [get_ports out] -clock clk
```

Note: For 5ns, change the clock period to 5ns in .sdc file

Step 4: Static Timing Analysis for the design with OpenSTA

Invoke OpenSTA in the terminal using the command: `sta`

4:1 Use the following commands:

Loads the liberty to osu018 standard cell library

i) `read_liberty osu018_stdcells.lib`

Reads your synthesized gate-level Verilog netlist

ii) `read_verilog d_ff_synth.v`

Links the loaded netlist and library to the top module name

iii) `link_design d_ff`

Loads your timing constraints from an SDC file

iv) `read_sdc d_ff.sdc`

Runs timing analysis to check all timing constraints and generates a report.

v) `report_checks`

OBSERVATION:

- Detect setup and hold timing violations
- Examine input and output delay constraints
- Review total negative slack and worst slack reports

OUTPUT:

Reports:

For 10ns:

➔ **for setup**

```
Startpoint: _4_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: q (output port clocked by clk)
Path Group: clk
Path Type: max

  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _4_/CLK (DFFPOSX1)
   3.68    3.68 ^ _4_/Q (DFFPOSX1)
   0.00    3.68 ^ q (out)
           3.68   data arrival time

  10.00   10.00   clock clk (rise edge)
   0.00   10.00   clock network delay (ideal)
   0.00   10.00   clock reconvergence pessimism
  -5.00    5.00   output external delay
           5.00   data required time
---------------------------------------------------------
           5.00   data required time
          -3.68   data arrival time
---------------------------------------------------------
           1.32   slack (MET)
```

➔ **for hold**

```
OpenSTA> report_checks -path_delay min
Startpoint: d (input port clocked by clk)
Endpoint: _4_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

   Delay     Time    Description
---------------------------------------------------------
    0.00     0.00    clock clk (rise edge)
    0.00     0.00    clock network delay (ideal)
    0.00     0.00 ^  input external delay
    0.00     0.00 ^  d (in)
    0.17     0.17 v  _2_/Y (INVX1)
    0.10     0.27 ^  _3_/Y (NOR2X1)
    0.00     0.27 ^  _4_/D (DFFPOSX1)
             0.27    data arrival time

    0.00     0.00    clock clk (rise edge)
    0.00     0.00    clock network delay (ideal)
    0.00     0.00    clock reconvergence pessimism
             0.00 ^  _4_/CLK (DFFPOSX1)
    0.01     0.01    library hold time
             0.01    data required time
---------------------------------------------------------
             0.01    data required time
            -0.27    data arrival time
---------------------------------------------------------
             0.27    slack (MET)
```

For 5ns:

➔ **for setup**

```
OpenSTA> report_checks
Startpoint: _4_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: q (output port clocked by clk)
Path Group: clk
Path Type: max

   Delay     Time    Description
---------------------------------------------------------
    0.00     0.00    clock clk (rise edge)
    0.00     0.00    clock network delay (ideal)
    0.00     0.00 ^  _4_/CLK (DFFPOSX1)
    3.68     3.68 ^  _4_/Q (DFFPOSX1)
    0.00     3.68 ^  q (out)
             3.68    data arrival time

    5.00     5.00    clock clk (rise edge)
    0.00     5.00    clock network delay (ideal)
    0.00     5.00    clock reconvergence pessimism
   -5.00     0.00    output external delay
             0.00    data required time
---------------------------------------------------------
             0.00    data required time
            -3.68    data arrival time
---------------------------------------------------------
            -3.68    slack (VIOLATED)
```

➔ **for hold**

```
Startpoint: d (input port clocked by clk)
Endpoint: _4_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

   Delay     Time    Description
---------------------------------------------------------
    0.00     0.00    clock clk (rise edge)
    0.00     0.00    clock network delay (ideal)
    0.00     0.00 ^  input external delay
    0.00     0.00 ^  d (in)
    0.17     0.17 v  _2_/Y (INVX1)
    0.10     0.27 ^  _3_/Y (NOR2X1)
    0.00     0.27 ^  _4_/D (DFFPOSX1)
             0.27    data arrival time

    0.00     0.00    clock clk (rise edge)
    0.00     0.00    clock network delay (ideal)
    0.00     0.00    clock reconvergence pessimism
             0.00 ^  _4_/CLK (DFFPOSX1)
    0.01     0.01    library hold time
             0.01    data required time
---------------------------------------------------------
             0.01    data required time
            -0.27    data arrival time
---------------------------------------------------------
             0.27    slack (MET)
```

COMPARISION:

Timing	Setup	Hold
10ns	1.32	0.27
5ns	-3.68	0.27

Note: Timing can be met by changing constraints in sdc.

CONCLUSION:

OpenSTA verifies the timing of the synthesized D flip-flop design using gate-level netlist and timing constraints. The analysis confirms that all critical paths meet setup and hold requirements, with positive slack values. No timing violations are detected, ensuring reliable operation at the specified clock frequency. The clock, reset, and counter increment logic show correct timing relationships in the reports. Overall, OpenSTA validates that the hardware counter will function accurately in real digital systems.